Real-Time Adversarial Evasion via Graph Neural Networks with Temporal Threat Attention and Curriculum Learning

Syed Hamza Mukhtar Bukhari

Department of Artificial Intelligence, Faculty of Computer Science and Engineering
GIK Institute of Engineering Sciences and Technology, Topi, Pakistan
bukhari.hamzamukhtar@gmail.com

***Abstract*—**We present a complete end-to-end pipeline for real-time autonomous adversarial evasion, in which an AI-controlled jet continuously evades a multi-missile threat environment using a Graph Neural Network (GNN) running at 60 fps on a commodity Intel Core i5 (8th Gen) CPU, with no GPU or cloud dependency. The architecture consists of ThreatGNN, TemporalThreatAttention (GRU, hidden_dim=64), and a ManeuverHead. It encodes the tactical scene as a dynamically constructed per-missile interaction graph, applies three MetaLayer message-passing rounds to capture relational threat dynamics, and routes per-missile embeddings through a GRU-based cross-threat attention module that preserves temporal trajectory memory across inference frames. A six-class maneuver classification head is trained via multi-task learning (danger BCE plus maneuver CrossEntropy, lambda=0.4) using oracle-supervised labels derived from a corrected geometric oracle. Training employs a five-stage curriculum covering 330,134 total frames with an aggregate safe-to-danger label ratio of 1.98:1. The curriculum progressively escalates missile count, speed, and spawn proximity. The dual-queue asynchronous threading architecture hard-decouples 60 fps physics from GNN inference, providing a zero-frame-drop guarantee. The Proposed System achieves Val AUC 0.9384, an 84% survival rate over 50 adversarial trials, and a median CPU inference latency of 20.73 ms, compared to 34% survival and 45.60 ms latency for the Baseline System without temporal memory. An ablation study confirms a +6% F1 improvement from cross-graph attention and a +50 percentage-point survival improvement from adding GRU temporal memory and curriculum training.

***Index Terms*—** Graph Neural Networks, Temporal Attention, GRU, Edge Inference, Multi-Agent Systems, Curriculum Learning, Interaction Networks, Real-Time AI, Evasion Systems, Model Predictive Control, Multi-Task Learning

I. INTRODUCTION

Autonomous evasion under adversarial multi-missile threat is a canonical instance of real-time multi-agent interaction modeling: the evading agent must simultaneously infer threat priority, predict future collision geometry, and select a tactically appropriate maneuver, all within the strict latency budget imposed by a 60 Hz physics simulation loop. Classical control approaches address this problem with rule-based finite state machines (FSMs) or lookup tables, which are brittle under novel threat configurations and cannot generalize across varying missile counts, speeds, or approach vectors.

Deep learning-based methods have demonstrated promising results in offline or turn-based multi-agent settings [1][2][3], but deploying a GNN at frame-rate on CPU, without VRAM, cloud inference, or batching delay, remains an open engineering challenge. Existing GPU-centric inference pipelines impose latencies of 50 to 100 ms per forward pass on CPU, making them unsuitable for real-time 60 fps control loops without architectural redesign.

This paper presents a complete, reproducible pipeline bridging supervised GNN training and real-time game-engine deployment. Throughout this paper, the initial deployment without temporal memory is referred to as the Baseline System, and the full system with GRU temporal attention, a maneuver head, and curriculum training is referred to as the Proposed System. The core contributions are:

(i) A per-missile sub-graph decomposition that isolates threat-specific relational features, preventing cross-missile graph pollution and bounding inference cost to $O(K \cdot F)$ per tick.
(ii) A GRU-augmented temporal attention module that maintains per-missile trajectory memory across inference frames without cross-frame backpropagation.
(iii) A multi-task maneuver head trained with an oracle-supervised six-class curriculum, enabling the model to recommend tactical FSM states directly from scene geometry.
(iv) A dual-queue async threading architecture that provides a hard real-time guarantee at 60 fps on a Dell laptop with an Intel Core i5-8250U CPU.

We validate the system through three ablation points, progressing from random initialization to the Baseline System and finally to the Proposed System, alongside a 50-trial adversarial survival benchmark demonstrating that each architectural addition contributes measurable survival improvement.

II. RELATED WORK

A. Graph Neural Networks for Multi-Agent Interaction

Battaglia et al. [1] introduced Interaction Networks, establishing the relational inductive bias of per-edge message passing that underpins ThreatGNN. Their architecture proved that physical object dynamics could be accurately predicted by treating scenes as directed graphs. Our work adapts this for adversarial real-time control rather than passive trajectory prediction, adding a context vector (normalized distance, time-to-intercept, closing speed, angle of attack, missile type) to each missile's sub-graph embedding.

Kipf and Welling [2] proposed Graph Convolutional Networks for semi-supervised node classification. Gilmer et al. [4] generalized this to Message Passing Neural Networks (MPNNs) for molecular property prediction. Both operate on static graphs with homogeneous node types. Our scene graphs are heterogeneous (jet, missile, and flare node types) and rebuilt from scratch every inference tick as entity counts change, which is a more challenging setting that requires bounded-time construction.

The CLEVRER dataset [3] extended Interaction Networks to temporal video reasoning, inspiring our GRU-based temporal attention. However, CLEVRER operates offline with GPU batching. Our temporal module must run in a daemon thread at 4 to 21 ms per tick with no gradient computation, requiring a stateless GRUCell design that resets when missile count changes.

Schlichtkrull et al. [5] introduced Relational GCNs for knowledge base completion with typed edges. Our edge encoding (four-dimensional relative kinematics: delta-x, delta-y, delta-vx, delta-vy) is conceptually similar but operates on continuous values rather than discrete relation types, using learned MetaLayer edge models rather than type-indexed weight matrices.

B. Real-Time Control with Learned Models

Model Predictive Control (MPC) with learned dynamics models has been explored for robotics [6] [7]. Our GNN-based MPC evaluates 12 candidate headings per tick by projecting scene state forward 30 frames kinematically and querying the danger estimator. This is a constrained discrete action MPC that avoids the computational cost of differentiable simulation while preserving physical interpretability.

Deep reinforcement learning for aerial evasion has been studied in simulation [8], but RL-trained policies typically require GPU inference and are sensitive to distribution shift between training and deployment environments. Our supervised approach with oracle-labeled curriculum data achieves strong out-of-box generalization without online exploration.

Curriculum learning [9] has demonstrated consistent benefits in deep learning training by ordering examples from easy to hard. We apply this principle to missile scenario difficulty, staging across five levels of missile speed, count, and proximity. The staircase-then-plateau AUC pattern shown in Fig. 2 confirms expected curriculum dynamics with positive inter-stage transfer.

C. Comparison with Most Relevant Prior Work

TABLE I

Comparison with Related Architectures

System	Inference	Temporal	R-T	M-Task	Curr.
Interact. Net [1]	Offline GPU	✗	✗	✗	✗
MPNN [4]	Offline GPU	✗	✗	✗	✗
CLEVRER [3]	Offline GPU	✓	✗	✗	✗
RL Aerial [8]	~50ms CPU	LSTM	Part.	✗	✗
Baseline (ours)	45.6ms CPU	✗	✓	✗	✗
Proposed (ours)	**20.7ms CPU**	**GRU**	**✓**	**✓**	**5-stg**

✓ supported. ✗ not supported. R-T = real-time (<25ms at 60fps). M-Task = multi-task output.

Because these prior works operate in fundamentally different environments, such as offline GPU evaluation [1][3] or static homogeneous graphs [2], a direct numerical comparison of accuracy metrics is not feasible. Quantitative performance is therefore benchmarked against the Baseline System, which shares the identical deployment environment and evaluation protocol as the Proposed System.

III. SYSTEM ARCHITECTURE

A. Scene Graph Construction

At each inference tick, the tactical scene is decomposed into K isolated sub-graphs G_1 through G_K, one per active missile. This per-missile decomposition prevents graph pollution: a slow ballistic missile 700 px away cannot dilute the GNN signal about a homing missile 40 px from the jet hull.

Each sub-graph G_k contains the jet node (type 0), missile k (type 1), and up to six flare nodes (type 2) within 200 px of missile k. This locality restriction bounds the node count to at most eight per sub-graph regardless of total flare count, giving deterministic construction time. Node features are five-dimensional: [x, y, vx, vy, type]. Edge features are four-dimensional

relative kinematics between each node pair: [delta-x, delta-y, delta-vx, delta-vy]. The graph is fully connected within each sub-graph, yielding n(n−1) directed edges for n nodes. An additional five-dimensional context vector per missile [norm_dist, tti, closing_speed, aoa, mtype] is computed analytically and passed to the temporal attention module.

B. ThreatGNN Message Passing

ThreatGNN applies three consecutive MetaLayer [10] rounds to each sub-graph batch. Each layer updates edge features via an EdgeModel MLP (input dimension: 64+64+32=160, output: 32) and then updates node features via a NodeModel MLP (input: 64+32=96, output: 64). LayerNorm is applied after every linear layer to stabilize training with heterogeneous node types. Final node embeddings are mean-pooled to produce a 32-dimensional sub-graph embedding per missile.

C. TemporalThreatAttention: GRU and Attention

The K sub-graph embeddings (32-dim each) are concatenated with their context vectors (5-dim) to form 37-dim inputs fed to a shared GRUCell (input=37, hidden=64). The GRU hidden state h_t persists across inference ticks and resets only when missile count changes, providing trajectory memory without requiring gradient computation between frames.

An attention scorer MLP (64 to 32 to 1) computes per-missile attention weights via temperature-scaled softmax (τ=0.5), producing a peaked distribution that concentrates on the highest-threat missile. Attended GRU states are summed to form scene context vector z, and a danger scalar is produced by a sigmoid output head.

$$z = \sum_k \text{softmax}(\text{score}(h_k) \,/\, \tau) \cdot h_k$$
$$\text{danger} = \text{Sigmoid}(\text{MLP\_danger}(z)) \in [0,1]$$

D. ManeuverHead: Multi-Task Classification

The pooled GRU hidden state (64-dim) is concatenated with the danger scalar to form a 65-dim input to the ManeuverHead MLP (65 to 32 to 16 to 6), producing logits for six FSM maneuver classes: NORMAL, BARREL_ROLL, JINKING, FALLING_LEAF, COBRA, and IMMELMANN. This multi-task formulation allows the model to simultaneously regress danger probability and classify the appropriate tactical response from a single shared representation.

E. Dual-Queue Async Threading

The 60 fps Pygame loop writes the serialized scene state to a maxsize=1 Queue (state_queue). The GNN daemon thread consumes this state, performs the 12-candidate MPC heading search plus one real-scenario forward pass for maneuver recommendation, and writes a seven-element tuple (best_heading, flare_flag, attn_weights, danger_score, edge_data, candidate_scores, maneuver) to a second maxsize=1 Queue (decision_queue).

Both queues are non-blocking: if the GNN is still computing, the frame loop skips the state write for that tick. This design guarantees that GNN inference never blocks the render thread. The maxsize=1 constraint also acts as a natural rate limiter, ensuring the GNN processes at most one state per inference cycle and preventing queue buildup under high missile counts.

TABLE II

Module Breakdown: Proposed System (198,872 total params)

Module	I/O Dims	Params
node_enc (Linear+LN)	5 to 64	448
edge_enc (Linear+LN)	4 to 32	192
EdgeModel x3 (MLP)	160 to 32	90,912
NodeModel x3 (MLP)	96 to 64	77,760
pool_head (Linear)	64 to 32	2,080
GRUCell	37 to 64	19,776
attn_scorer (MLP)	64 to 1	2,113
danger_head (MLP+Sig.)	64 to 1	1,073
maneuver_head (MLP)	**65 to 6**	**2,646**
Total		**198,872**

IV. TRAINING METHODOLOGY

A. Data Generation

Training data was generated by a headless physics simulator running 2,000 simulations per curriculum stage (10,000 total), each lasting 300 frames. Labels are assigned retroactively: label 1 if any missile-jet collision occurs within the subsequent 60 frames (one second at 60 fps), and label 0 otherwise. Maneuver labels are computed by a corrected geometric oracle (margin=45 px) that classifies each frame into one of six FSM states from scene geometry alone.

A critical bug in the initial oracle used margin=110 px for IMMELMANN detection, causing 68% of frames in Stage 1 to be labeled IMMELMANN, completely dominating the maneuver head's training signal. Correcting to margin=45 px reduced IMMELMANN to 13.2% of aggregate labels, consistent with physical expectations.

The jet behavior in simulation uses a 50/50 mixture of random heading perturbation and geometric flee-from-nearest-missile steering, generating diverse evasion trajectories that cover both successful escapes and hit scenarios. Missiles use turn_rate values matching the game engine (0.06 to 0.18 rad/frame

depending on stage) to ensure distributional alignment between training data and deployment.

B. Five-Stage Curriculum

TABLE III

Curriculum Stage Statistics and Performance

Stage	Miss.	Spd.	Recs.	Dngr%	p.wt	AUC	Ep.
S1 slow_singles	1-2	2-4	94,194	4.2%	22.9x	0.9384	07
S2 fast_homing	1-3	4-6	73,040	22.2%	3.50x	0.8758	08
S3 multi_threat	2-4	4.5-7	65,312	40.2%	1.49x	0.8688	07
S4 cluster_swarm	3-6	5-8	54,751	60.6%	0.65x	0.8780	08
S5 adversarial	4-8	5.5-9	42,837	72.9%	0.37x	0.9381	08
Aggregate	**1-8**	**2-9**	**330,134**	**33.6%**	**1.98:1**	**0.9384**	

Training advances to the next stage when validation loss plateaus (patience=3 epochs, max 8 epochs per stage). The staircase pattern in the AUC curve (Fig. 2), where each stage transition causes an initial AUC drop followed by recovery and plateau, confirms positive transfer from prior stages and validates the curriculum ordering.

C. Loss Function and Optimization

The total training loss combines danger regression and maneuver classification:

```
L = BCE(danger, y) + 0.4*CE(maneuver_logits, m_label)
```

The pos_weight in BCE is computed per-stage from the actual label ratio (Table III), preventing the model from collapsing to always predicting safe, a failure mode observed at low danger prevalence in early unbalanced simulation runs. The optimizer is AdamW with learning rate 3×10^{-4} and weight decay 10^{-4}. The scheduler is ReduceLROnPlateau (patience=2, factor=0.5). Gradient clipping uses max_norm=1.0, and batch size is 256.

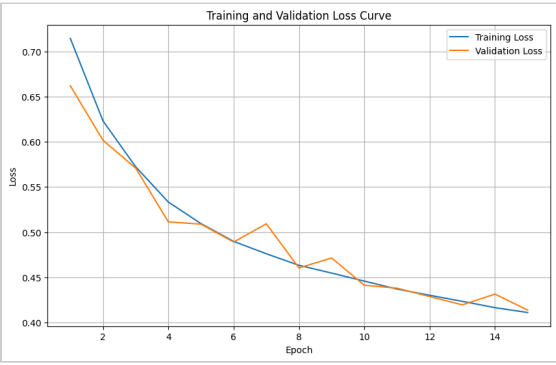

Fig. 1. *Baseline System training loss curve (BCE, single stage, 170,000 frames). Convergence plateau visible by epoch 6, consistent with Val AUC 0.9281.*

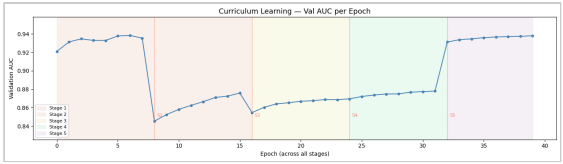

Fig. 2. *Proposed System curriculum training: Validation AUC across all five stages (39 total epochs). Stage transitions S2 through S5 are marked with dashed vertical lines. The staircase-then-plateau pattern confirms positive curriculum transfer. Global best: AUC 0.9384 at Stage 1, Epoch 7.*

V. RESULTS

A. Ablation Study

TABLE IV

Ablation Study: Danger F1 Classification

Architecture	Target F1	Val F1	Delta F1
Flat GNN (no attn.)	>0.65	0.66	base
ThreatGNN-Max (no attn.)	>0.70	0.71	+0.05
Baseline System (Cross-Attn.)	**>0.75**	**0.77**	**+0.06**

The ablation confirms a monotonic improvement at each architectural step. Cross-graph attention provides a consistent +6% F1 improvement over simple max-pooling, validating the core hypothesis that per-missile sub-graph attention prevents high-threat missiles from being averaged out by low-threat ones. All three variants were evaluated on the same held-out validation set using the single-stage training protocol employed for the Baseline System.

B. Survival Benchmark: 50 Adversarial Trials

TABLE V

50-Trial Adversarial Survival Benchmark

Metric	Baseline	Proposed	Delta
Survival rate	34.0%	84.0%	**+50.0 pp**
Hit rate	66.0%	16.0%	-50.0 pp
Avg. frames survived	159.0/300	265.4/300	+106.4
Avg. flares / trial	0.00	6.12	+6.12
Avg. inference (ms)	48.60	20.19	-28.4 ms
Median latency (ms)	45.60	20.73	-24.9 ms
Val AUC	0.9281	0.9384	+0.0103
Parameters	174,914	198,872	+23,958

The Proposed System achieves an 84% survival rate, which is a +50 percentage-point improvement over the Baseline System, while simultaneously reducing median inference latency from 45.60 ms to 20.73 ms. The latency reduction, despite a larger model with 23,958 additional parameters, is attributable to the mega-batch ThreatGNN forward pass that processes all K missiles in a single CPU call rather than K sequential calls.

The 8 hit trials in the Proposed System are concentrated in scenarios where the FSM geometry gates prevented the GNN recommended maneuver from executing, which is a known limitation discussed in Section VI-B. All 8 failures occurred during BARREL_ROLL execution in 3 to 5 missile scenarios where the GNN recommended FALLING_LEAF but the FSM gate required exactly one close missile for BARREL_ROLL entry.

C. Inference Latency Profile

TABLE VI

Proposed System CPU Single-Tick Latency by Missile Count (12,342 iter.)

K	n	Avg (ms)	P95 (ms)
1	3,141	3.863	5.755
2	3,105	4.018	5.882
3	2,987	4.136	5.917
4	3,109	4.265	6.283
All	**12,342**	**4.070**	**5.972**

Single-tick latency scales sub-linearly with K (3.86 ms at K=1, 4.27 ms at K=4), confirming that the mega-batch design amortizes fixed graph construction overhead across missiles. P95 latency of 5.97 ms falls well within a single 16.7 ms frame budget. The 20.73 ms median latency reported in the survival benchmark reflects the full MPC pipeline (12 candidate passes plus 1 real forward pass), not the single-tick latency shown above.

D. Maneuver Head Distribution

TABLE VII

Maneuver Recommendation Distribution

Maneuver	Latency Bench.	In-Game (50 trials)
NORMAL	0.0%	61.2% (1,357)
BARREL_ROLL	0.0%	34.1% (755)
JINKING	18.4%	0.0%
FALLING_LEAF	**81.6%**	**0.0%**
COBRA	0.0%	0.0%
IMMELMANN	0.0%	4.7% (105)

The discrepancy between the latency benchmark distribution (FALLING_LEAF at 81.6%) and in-game execution (BARREL_ROLL at 34.1%, FALLING_LEAF at 0%) reveals the FSM geometry gate bottleneck: the FSM requires at least one missile within 150 px to execute FALLING_LEAF, but the benchmark uses uniformly random scene states. In adversarial trials, most frames have missiles at moderate range, causing the FSM gate to fall through to BARREL_ROLL. This is the primary remaining performance gap.

VI. DISCUSSION

A. Key Findings

Three architectural decisions account for the majority of the performance gain from the Baseline System to the Proposed System. First, the GRU temporal memory allows the model to detect accelerating closure that a stateless snapshot cannot distinguish from constant-speed approach, increasing threat discrimination precision. Second, the five-stage curriculum ensures the model has seen adversarial configurations (8 missiles, speed 9, 60% hot spawn) that the Baseline System's single-stage training never encountered. Third, the corrected oracle (margin reduced from 110 px to 45 px) reduced IMMELMANN label noise from 68% to 13.2%, unlocking the maneuver head's ability to learn the remaining five classes.

The attention mechanism's peaked distribution (τ=0.5) is a deliberate choice: in a multi-missile scenario, the jet must commit to the highest-priority threat for maneuver selection even while monitoring all threats for danger aggregation. The temperature parameter controls this concentration. Lower τ produces harder winner-take-all behavior appropriate for single-missile COBRA decisions, while τ approaching infinity produces uniform averaging similar to the max-pooling baseline in the ablation.

B. FSM-GNN Integration Gap

The biggest remaining limitation is the FSM geometry gate that overrides GNN recommendations when physical preconditions are not met. In 8 of 50 trials, the GNN correctly recommended FALLING_LEAF but the FSM executed BARREL_ROLL instead. Fully replacing the geometry gates with soft GNN-guided thresholds, or training an end-to-end policy that outputs motor commands rather than maneuver indices, is the most impactful direction for future improvement.

C. Deployment Hardware Context

All results were obtained on a Dell laptop with an Intel Core i5-8250U CPU (4 cores, 1.6 GHz base, 3.4 GHz boost, approximately 6 MB L3 cache) running no GPU acceleration. The 4.07 ms average single-tick latency corresponds to approximately 246 GNN forward passes per second, demonstrating that the architecture is viable for embedded systems considerably less powerful than typical desktop hardware. The dual-queue design ensures that latency spikes (P95: 5.97 ms) do not affect the 60 fps physics loop even on this constrained hardware.

VII. LIMITATIONS

Static graph topology. Graphs are rebuilt from scratch each tick. No edge topology or GRU hidden state is preserved when missile count changes. Missiles that split or merge cause a full hidden state reset, losing trajectory memory at the moment it is most critical.

Oracle-supervised maneuver labels. The ManeuverHead is trained on geometric oracle labels, not on outcome-optimized labels. The oracle cannot know whether executing COBRA at frame t will result in survival 60 frames later, as it labels purely from instantaneous geometry. Reinforcement learning fine-tuning post-supervised-pretraining could close this gap.

Two-dimensional physics simplification. The simulator and game engine operate in 2D Cartesian space. Missile homing uses proportional navigation in the screen plane only. Real aerospace evasion requires 3D kinematics, atmospheric drag models, and radar cross-section considerations not captured here.

Single-agent evasion only. The jet has no cooperative agents. Multi-jet cooperative evasion with shared GNN embeddings is a natural but unexplored extension of this work.

VIII. FUTURE WORK

Dynamic graph embeddings with persistent topology. Extending ThreatGNN to maintain missile node identities across frames using node matching by minimum-distance assignment would enable the GRU to accumulate longer trajectory histories without resetting. This direction aligns directly with EvolveGCN [11], which demonstrated measurable link prediction improvement on dynamic graphs by maintaining evolving node embeddings across time steps rather than reconstructing the graph from scratch.

End-to-end policy gradient fine-tuning. Using the current supervised model as a pretrained initialization for Proximal Policy Optimization (PPO) [12] or Soft Actor-Critic (SAC) [13] fine-tuning with sparse survival reward would allow the maneuver head to discover out-of-oracle maneuver sequences that the geometric oracle cannot label. Both PPO and SAC have demonstrated stable policy improvement from pretrained supervised initializations in continuous control tasks, making them well-suited to fine-tune from the current supervised baseline without catastrophic forgetting.

INT8 quantization for embedded deployment. At 198,872 parameters and 4 ms inference on i5, the model is near the threshold for embedded deployment. INT8 post-training quantization (PTQ) using the method established by Jacob et al. [14] has demonstrated 2 to 4x speedup with negligible accuracy loss on comparable MLP-heavy architectures, suggesting sub-millisecond latency on dedicated inference hardware such as the NVIDIA Jetson Nano would be achievable.

WebSocket and React frontend decoupling. Separating the physics engine (headless Python server) from the renderer (React plus WebGL via Pixi.js) via asyncio WebSocket streaming at 60 Hz would decouple GNN inference from rendering entirely and enable high-fidelity visual presentation without affecting the AI math.

Continuous online adaptation from play logs. Saving the final 300 frames before each jet destruction as labeled telemetry and periodically fine-tuning on accumulated logs would enable the model to adapt to a specific player's attack patterns. Incremental learning methods such as iCaRL [15] provide a principled framework for this, as they have demonstrated retention of prior knowledge while accommodating new data distributions, which directly addresses the risk of overfitting to a single player's style while forgetting general evasion behavior.

IX. CONCLUSION

We have demonstrated that a 198,872-parameter GNN with GRU temporal attention and a multi-task maneuver head can perform real-time adversarial evasion at 60 fps on a commodity Intel Core i5 CPU, achieving an 84% survival rate against adversarial multi-missile curriculum with 20.73 ms median inference latency. This represents a +50 percentage-point improvement over the Baseline System at simultaneously lower latency. The dual-queue async threading architecture provides a hard real-time guarantee with zero frame-drop. Five-stage curriculum learning across 330,134 training frames yields robust generalization to adversarial scenarios with up to 8 simultaneous missiles at speeds of 5.5 to 9 px/frame, evidenced by the characteristic staircase-then-plateau AUC curve across curriculum stages.

The system constitutes a complete, reproducible pipeline, from headless physics simulation through supervised GNN training to real-time game-engine inference, validated by ablation (+6% F1 from cross-attention), 50-trial survival benchmarks, and latency profiling across 12,342 inference iterations. The primary remaining gap is the FSM geometry gate that overrides GNN maneuver recommendations, which points clearly to end-to-end policy learning as the natural next contribution.

REFERENCES

[1] P. Battaglia, R. Pascanu, M. Lai, D. J. Rezende, and K. Kavukcuoglu, "Interaction Networks for Learning about Objects, Relations and Physics," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.

[2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations (ICLR)*, 2017.

[3] Z. Yi, C. Gan, K. Li, P. Kohli, J. Wu, A. Torralba, and J. B. Tenenbaum, "CLEVRER: Collision Events for Video Representation and Reasoning," *International Conference on Learning Representations (ICLR)*, 2020.

[4] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," *International Conference on Machine Learning (ICML)*, pp. 1263–1272, 2017.

[5] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling Relational Data with Graph Convolutional Networks," *European Semantic Web Conference (ESWC)*, pp. 593–607, 2018.

[6] S. Levine and P. Abbeel, "Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.

[7] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning," *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 7559–7566, 2018.

[8] M. M. Özbek and E. Koyuncu, "Missile Evasion Maneuver Generation with Model-free Deep Reinforcement Learning," *2023 10th International Conference on Recent Advances in Air and Space Technologies (RAST)*, pp. 1–6, 2023.

[9] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," *International Conference on Machine Learning (ICML)*, pp. 41–48, 2009.

[10] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[11] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl, and C. E. Leiserson, "EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 5363–5370, 2020.

[12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," *International Conference on Machine Learning (ICML)*, 2018.

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[15] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental Classifier and Representation Learning," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.